A Verifiable Secure Aggregation for Federated Learning with Low-quality Data

[a]*School of Computer and Software Engineering, Xihua University, Chengdu 610039, China*

Abstract

Federated learning lets multiple participants build a shared model collaboratively while their raw data remain entirely local, and it has shown great potential in various fields. Many studies focus on the security of federated learning to prevent incorrect results return and user data leakage. However, low-quality data are overlooked, which may render the trained global model unusable. To fill this gap, we devise a privacy-preserving and verifiable weighted-aggregation protocol that alleviates the adverse influence of low-quality data on the global model. In the scheme, users' gradients and the weights of gradients are protected by using secret sharing and truth discovery techniques respectively. Meanwhile, the aggregation results computed by the cloud server can be verified. Finally, we conduct simulation experiments on our scheme to demonstrate its feasibility.

Keywords: Federated Learning, Secure Aggregation, Low-quality Data, Verifiable Computation.

1. Introduction

As data-driven applications proliferate, safeguarding personal information has become increasingly imperative. With the continuous collection and processing of personally sensitive information, the risk of data breaches has risen substantially. In response, governments and regulatory organizations have introduced stringent data privacy regulations. For instance, the European Union's General Data Protection Regulation (GDPR) requires minimizing personal data processing and strictly prohibits unauthorized usage

*Corresponding author

[1]. These regulatory initiatives have exerted a profound influence on traditional machine learning methods. Traditional machine learning requires merging large datasets into centralized servers to leverage the data's potential. This centralized paradigm enables predictive models by exploiting comprehensive datasets, enhancing model performance and accuracy. However, this methodology poses challenges when dealing with sensitive data such as healthcare records, financial information, or personal identifiers. In 2016, Google proposed federated learning [2] to address these issues. Federated learning trains models collaboratively on edge devices, transmitting only parameter updates—such as gradients or weights—while raw data stay local. By decentralising computation, federated learning both preserves confidentiality and scales efficiently over heterogeneous data sources.

Despite federated learning's privacy advantages, researchers have shown that exchanged gradients can still leak information through membership-inference [3] and model-inversion attacks [4]. On the other hand, data quality varies among users in real-life scenarios: high-end device users usually generate high-quality data, while low-performing device users produce low-quality data. Studies [5–7] indicate that low-quality data can slow model convergence or render the model useless during training. In addition, a malicious server might cut corners during aggregation or deliberately falsify results, thereby skewing the learned model [8].

In this paper, we focus on the following question: *How can we construct a federated learning scheme that can effectively protect users' privacy, efficiently utilize data (including low-quality data), and verify the correctness of the aggregated results generated by the cloud server?* Therefore, we propose a verifiable secure weighted aggregation federated learning scheme. The proposed scheme has three properties: (i) mitigates the impact of unreliable users by assigning higher weights to high-quality data, (ii) employs masking to protect both the weighted gradients and the corresponding weight values, and (iii) leverages the additive homomorphism of secret sharing to verify the aggregation results.

The protocol matches deployments where privacy, untrusted aggregation, and heterogeneous data coexist, such as mobile keyboard prediction at scale, multi-institution medical imaging, and financial analytics. These settings have already applied federated learning in practice, and our single-server, verifiable, reliability-weighted design aligns with such constraints.

Our contributions can be outlined as follows:

- **Secure and verifiable single-server aggregation.** We design a single-server protocol that protects updates and lets users verify each aggregation with exact field arithmetic and a threshold secret-sharing backbone, while tolerating drop-outs.
- **Privacy-preserving reliability weighting.** Weighted gradients and weights are protected end-to-end by double masking and secret sharing; the reliability weighting is computed locally and incorporated without leaking per-user values.
- **Deployability.** The workflow supports partial participation under a single untrusted server, without relying on any helper server.

The rest of the paper is structured as follows: Section 2 reviews related work; Section 3 outlines preliminaries; Section 4 formalizes the problem statement; Section 5 details our method; Section 6 presents security analyses. Section 7 reports the experimental evaluation; and Section 8 concludes.

2. Related Work

2.1. Verifiable Federated Learning

In federated learning, malicious cloud servers might provide inaccurate aggregation results to users. Therefore, users are encouraged to validate the accuracy of the aggregation results returned by the cloud server. Xu et al. [9] introduced Verifynet, which secures gradient vectors through a combination of homomorphic hashes and pseudorandom functions. Unlike VerifyNet, VeriFL [10] requires only the hash of each user's gradient,greatly reducing communication overhead and making it suitable for resource-constrained scenarios. However, its verification ability may be limited by the characteristics of the hash function. Similarly, VFL [11] protects user gradients by leveraging Lagrange interpolation, pseudorandom functions, and the Chinese Remainder Theorem. In [12], Xu et al. proposed a non-interactive verifiable federated learning scheme by combining Lagrangian set interpolation polynomials and a two-server architecture. Similarly, Cai et al. [13] devised a cross-validation method in which verification occurs without any interaction among users. Xia et al. [14] proposed SVCA, which couples commitment schemes with Hadamard products to ensure both the correctness of aggregation and the security of verification.

2.2. FL with low-quality Data

However, above schemes could not address the challenge posed by low-quality data. Only a few approaches have effectively combined privacy preservation with mechanisms for low-quality data. For example, Zhao et al. [6] introduced SecProbe, applying differential privacy to perturb the objective model's loss function. Yet, doubts persist about its ability to balance privacy protection and utility. Subsequently, Xu et al. [7] put forward $Meth_{IU}$ method, integrating Yao's garbled circuit with Paillier's homomorphic cryptosystem to safeguard user privacy and mitigate the influence of low-quality data on the global model. The protocol requires non-cooperation between two servers. More recently, EPPFL [5] used a threshold Paillier cryptosystem to safeguard user gradients and proposed a framework to mitigate the impact of unreliable users.

2.3. Application-driven deployments

Real-world deployments indicate that production FL must satisfy stricter requirements than benchmark settings, including privacy at scale, partial participation, and tight communication budgets [15]. A production system report for mobile, cross-device FL details a scalable orchestration stack with secure aggregation and tolerance to client drop-outs [15]. In healthcare, a 10-institution medical-imaging study showed that collaborative training can approach centralized performance and generalize to external sites while avoiding raw data sharing [16]. During the COVID-19 pandemic, a 20-hospital federated study trained prognostic models from clinical and imaging data, demonstrating operational feasibility under heterogeneous, unharmonized datasets and strict compliance constraints [17]. Together, these deployments motivate lightweight, verifiable, and reliability-aware protocols that fit practical constraints of cross-device and cross-silo settings.

3. Preliminaries

In this section, federated learning and truth discovery technologies are introduced first. We then give a concise overview of Shamir secret sharing and the Diffie-Hellman key exchange protocol.

3.1. Federated Learning

Federated learning is a distributed machine learning framework that typically involves a cloud server and N users u_i, where each user u_i holds a locally

stored private dataset, $i \in \{1, ..., N\}$. In each iteration $T \in \{1, 2, \dots\}$, the cloud server sends the current global gradient $\boldsymbol{x}^{T-1}$ to every participant. User u_i computes the model parameters $\boldsymbol{x}_i^T$ using the Stochastic Gradient Descent (SGD) algorithm on their sensitive dataset and sends the computed gradients back to the cloud server. Using the federated averaging algorithm [18], the cloud server aggregates all users' gradients to obtain the weighted average gradient $\boldsymbol{x}^T = \frac{1}{N} \sum_{i=1}^{N} \boldsymbol{x}_i^T$. It repeats this routine—aggregation and broadcast—until the target accuracy is achieved.

The above process assumes that the datasets of all users are of high quality and have similar distributions. However, low-quality data may reduce the convergence speed of the aforementioned model. Next, we will introduce truth discovery technology, which can assign different weights to data of different qualities, thus mitigating the adverse influence of low-quality samples on the final accuracy.

3.2. Truth Discovery

We will use the same truth discovery method as PPFDL [7] to compute the aggregation weights of different users' data. The aggregation weight w_i for user u_i is determined as follows:

$$w_i = \frac{\mathcal{C}}{F\left(\boldsymbol{x}_i^T, \boldsymbol{x}^{T-1}\right)}, \tag{1}$$

where $\mathcal{C}$ is the coefficient used to scale the user weights, $F\left(\boldsymbol{x}_i^T, \boldsymbol{x}^{T-1}\right)$ measures the distance between user u_i's local gradient $\boldsymbol{x}_i^T$ and the global gradient $\boldsymbol{x}^{T-1}$.

A shorter distance between $\boldsymbol{x}_i^T$ and $\boldsymbol{x}^{T-1}$ implies higher data quality for that user, and thus a larger weight w_i. As mentioned in the literature [19], user gradient $\boldsymbol{x}_i^T$ with smaller distances can be assigned larger aggregation weights w_i, which ensures that high-quality data mainly contribute to the global model.

After calculating the aggregation weights for all users, the global gradient is updated as follows.

$$\boldsymbol{x}^T = \frac{\sum_{i=1}^{n} w_i \boldsymbol{x}_i^T}{\sum_{i=1}^{n} w_i}. \tag{2}$$

3.3. Secret Sharing

Shamir secret sharing is a seminal threshold scheme that partitions a secret into multiple shares. The scheme comprises two core routines: secret distribution $SS.sh(\cdot)$ and secret reconstruction $SS.re(\cdot)$. The specific algorithms are as follows:

- $\{(\beta_i, f(\beta_i))\}_{u_i \in U} \leftarrow SS.sh(s, U, t)$: given a secret s, a set $U = \{u_i\}_{i \in [1,n]}$ of all users, and a threshold t. Randomly choose $t-1$ coefficients $a_i \in \mathbb{Z}_q$ to define a polynomial

$$f(x) = s + a_1 x + a_2 x + \cdots + a_{t-1} x^{t-1} \bmod q. \tag{3}$$

  The share of the secret s can be denoted as $\{(\beta_i, f(\beta_i))\}_{u_i \in U}$.

- $s \leftarrow SS.re\left(\{(\beta_i, f(\beta_i))\}_{u_i \in U}, t\right)$: given any t shares $(\beta_i, f(\beta_i))$, the secret s can be reconstructed utilizing the Lagrange interpolation method. The following formula can efficiently recover secrets

$$s = \sum_{j=0}^{t} f(\beta_j) \prod_{m \neq j}^{t} \frac{\beta_m}{\beta_m - \beta_j} \bmod q. \tag{4}$$

The Shamir secret sharing has additive homomorphism [20]. Assuming two users u_1 and u_2 respectively hold secrets s and s', user u_1 computes the shares set of s as $\{(\beta_i, f(\beta_i))\}_{u_i \in U}$ by $SS.sh(s, U, t)$, and user u_2 utilizes $SS.sh(s', U, t)$ to calculate the set of shares of s' to be $\{(\beta_i, f'(\beta_i))\}_{u_i \in U}$. Then the secret $s + s'$ can be reconstructed by:

$$s + s' \leftarrow SS.re(\{(\beta_i, f(\beta_i) + f'(\beta_i))\}_{u_i \in U}, t). \tag{5}$$

Shamir secret sharing is linear over the field F. Local addition of shares gives a share of the sum, and reconstruction returns the exact value in F. Therefore, repeated additions of shares do not accumulate noise; the interpolation only changes the underlying polynomial accordingly.

3.4. Diffie-Hellman Key Agreement

The Diffie-Hellman protocol enables two parties to establish a common secret key across a public channel without any prior exchange of confidential data. The resulting shared secret key can be used for subsequent encrypted communications. It involves three types of probabilistic polynomial-time algorithms:

- $pp \leftarrow KA.setup(1^{\lambda})$: given the security parameter λ, this routine outputs the public parameter pp.
- $(sk_u, pk_u) \leftarrow KA.gen\,(pp)$: using pp, the algorithm produces the user's private and public keys (sk_u, pk_u).
- $s_{u,v} \leftarrow KA.agree(sk_u, pk_v)$: combining user u's private key sk_u with user v's public key pk_v, the procedure derives the shared secret $s_{u,v}$.

Note that, $s_{u,v} = KA.agree(sk_u, pk_v)$ is equal to $s_{v,u} = KA.agree(sk_v, pk_u)$.

3.5. Symmetric Encryption

In symmetric cryptography, the same key serves for both encryption and decryption. A symmetric encryption system consists of three algorithms:

- KeyGen$(\lambda) \rightarrow k$: provided with security parameter λ, KeyGen returns a symmetric key k.
- Enc$(k, m) \rightarrow c$: given key k and plaintext m, Enc produces the ciphertext c.
- Dec$(k, c) \rightarrow m$: supplied with key k and ciphertext c, Dec recovers the plaintext m.

A symmetric encryption system satisfies the following correctness property: $Dec(k, Enc(k, m)) = m$.

4. Problem Statement

This section presents the system architecture and the associated threat assumptions of our scheme.

4.1. System Model

As shown in Fig.1, our system comprises three entities: trusted authority, cloud server, and users.

- **Trusted Authority (TA):** The TA act as an honest third party, tasked with parameter initialization and key-pair generation for every user. TA has secure channels with all users. It distributes the parameters and key pairs to users through the secure channel.

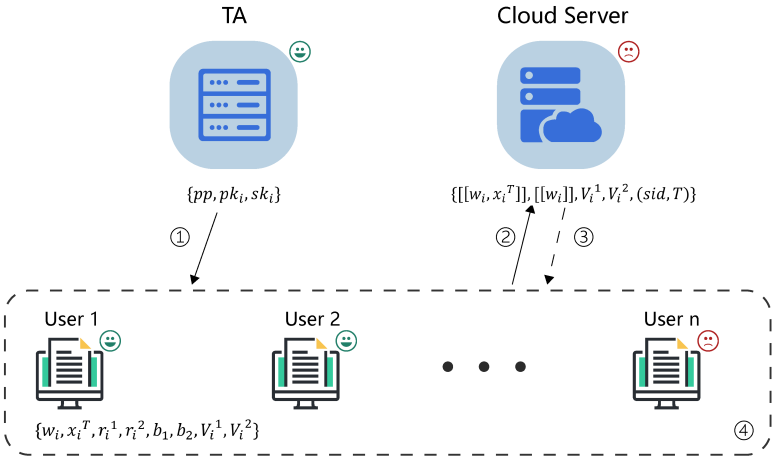

Figure 1: System Model (Legend: TA securely distributes pp and (pk_i, sk_i), then goes offline; pp means $\{\boldsymbol{x}^0, \boldsymbol{a}, B, c, d, sid\}$. Solid arrows mean uploads to server; dashed arrows mean server-relayed encrypted peer shares; dotted box means user trust boundary. $[\![\cdot]\!]$ means masked values; V_i^1, V_i^2 mean per-round verification tags. Numbers 1–4 mark the four phases: setup, local computation and upload, secure aggregation, verification and update. smile icon means benign and frown icon means malicious.)

- **Cloud Server:** The cloud server has sufficient computing resources but no training dataset. It can update the global model by combining users' gradients according to their assigned weights. It also compiles the users' verification labels and returns both the aggregated result and these verification labels to each user.

- **Users:** Each user has their own private dataset locally. Some users' data are of high quality, whereas others are of low quality. They can obtain locally updated gradients by running local model training on their private data. All users have a private authenticated channel with the cloud server. Subsequently, the encrypted gradients, the encrypted weights, and verification labels are transmitted to the cloud server through the authenticated channel. After the cloud server performs the aggregation, users obtain both the aggregation outcome and its verification labels for integrity checking.

4.2. Threat Model

Following the VerSA [8] threat model, the TA is fully trusted and never colludes with any participant. The cloud server is malicious, who attempts to

access users' private data. Furthermore, it could manipulate the aggregation results. All users are assumed to be honest-but-curious: they follow the protocol faithfully yet may attempt to infer other parties' private information. In addition, the users will not collude with the cloud server.

5. The Proposed Scheme

This section first briefly describes the process of the proposed scheme, and then provides a detailed description of the scheme.

5.1. Overview

Our proposed scheme has four phases: setup phase, local computation phase, secure aggregation phase, and gradient update phase.

In the setup phase, TA generates the system parameters pp and the key pair for each user (pk_i, sk_i), TA send pp, pk_i, sk_i to user u_i via the secure channel and does not retain any user's secret key. After delivery, TA goes offline. User u_i uploads pk_i to the cloud server. The cloud server collects the received public keys and broadcasts them to the online users U_1. To avoid seed reuse, a master seed B is included in pp For the T-th round, parties derive a fresh per-round seed $b^{(T)} = PRF(B, sid||T)$ to generate one-time masks and verification labels, where sid is the session identifier.

In the local computation phase, user in U_1 obtains his local gradient by local model training. Then users calculate the weights of their own gradients using equation (1). Next, user randomly selects two masks to encrypt the weighted gradient and weight respectively. Meanwhile, user splits the two masks by secret sharing. In addition, user computes the verification labels of weighted gradient and weight. Finally, user sends above message to the cloud server.

In the secure aggregation phase, the cloud server leverages the additive homomorphism of Shamir secret sharing to sum the weighted gradients and the corresponding weights. Because these computations are exact over the field F, they introduce no numerical noise; the outputs remain valid Shamir shares of the plaintext sums. The server also sums the users' verification labels so that each user can check the correctness of the aggregates. It then sends the aggregated values and the proofs to the online users.

In the gradient update phase, the online users can verify the results of the aggregated weighted gradient and the aggregated weights. If validation is

successful, the users will adopt the aggregated results and update the global gradient. Otherwise, they will abort the computation.

5.2. The Proposed Scheme

Let $\mathbb{F}$ be a field, n be an integer number, m be the dimension of gradient, and t be the threshold of secret sharing. The four phases mentioned above are described in detail as follows.

Phase 1: Setup phase

There are n users $U = \{u_i, i \in [1, n]\}$. TA samples a public session identifier $sid \in \{0,1\}^\lambda$ and generates the system parameters $pp = \{\boldsymbol{x}^0, \boldsymbol{a}, B, c, d, sid\}$, where $\boldsymbol{x}^0 \in \mathbb{F}^m$ is the initial gradient, $\boldsymbol{a} \in \mathbb{F}^m$ is a random vector, $B \in \mathbb{F}$ is the master seed, and $c, d \in \mathbb{F}$ are random numbers. For round T, parties derive a fresh seed $b^{(T)} = \mathrm{PRF}(B,\ sid \,||\, T)$ to generate one-time masks and verification labels, no seed is reused across rounds. TA generates (pk_i, sk_i) for each u_i, sends pp and (pk_i, sk_i) via the secure channel, keeps no copy of sk_i, and then goes offline. User u_i uploads pk_i to the cloud server. Let $U_1 \subseteq U$ be the set of users that submitted public keys; the server broadcasts these keys to all users in U_1.

For $T = 1$ users ignore any server-supplied initial model and use the x^0 embedded in pp, and sid is a public session identifier chosen at setup, it uniquely labels the training session and is included in all messages and PRF inputs to bind rounds and prevent cross-session replay.

Phase 2: Local computation phase

At T-th round, we assume that users have gained the $(T-1)$-th global gradient $\boldsymbol{x}^{T-1}$ from the cloud server.

Firstly, each user $u_i \in U_1$ obtains the gradient $\boldsymbol{x}_i^T$ by local model training. According the truth discovery technology, user u_i computes the weight of this round as follows:

$$w_i = \frac{\chi^2_{\left(1-\frac{\alpha}{2}\right),m}}{\left(\boldsymbol{x}_i^T - \boldsymbol{x}^{T-1}\right)^2}, \tag{6}$$

where χ^2 represents the chi-squared distribution, and α denotes the corresponding significance level. As illustrated in [21], local gradients trained with high-quality data are closer to the global gradient than those trained with low-quality data. As shown in the above formula (6), high-quality data

are assigned a higher weight, while low-quality data receive a lower weight. Therefore, the gradients aggregated by the cloud server will be more biased towards those from high-quality data, thereby reducing the negative impact of low-quality data on the global model. Therefore, the χ^2-based reliability weight in (6) concretely instantiates the truth-discovery idea introduced in Sec. 3.2.

Secondly, user u_i randomly selectes a vector $\boldsymbol{r}_i^1 \in \mathbb{F}^m$ and a value r_i^2 as mask to encrypt the weighted gradient $w_i \boldsymbol{x}_i^T$ and the weight w_i:

$$\llbracket w_i \boldsymbol{x}_i^T \rrbracket = w_i \boldsymbol{x}_i^T + \boldsymbol{r}_i^1 + \boldsymbol{b}_1, \tag{7}$$

$$\llbracket w_i \rrbracket = w_i + r_i^2 + b_2, \tag{8}$$

where $\boldsymbol{b}_1 \in \mathbb{F}^m$ and $b_2 \in \mathbb{F}$ are derived from the per-round seed $b^{(T)} = PRF(B, sid||T)$ using domain separation, e.g., $\boldsymbol{b}_1 = PRF(b^{(T)}, \texttt{"mask:x"})$ and $b_2 = PRF(b^{(T)}, \texttt{"mask:w"})$. Here, we employ a two-layer masking method to encrypt the weighted gradient. The first-layer mask $\boldsymbol{r}_i^1$ is used to hide the weighted gradient, while the second-layer mask $\boldsymbol{b}_1$ serves to protect the random number $\boldsymbol{a}$ within the subsequent verification tag $\boldsymbol{V}_i^1$. Similarly, the weight w_i is encrypted using the same idea.

Meanwhile, user u_i uses secret sharing to split the mask $\boldsymbol{r}_i^1$ and r_i^2, i.e.

$$(u_j, \boldsymbol{r}_{i,j}^1)_{u_j \in U_1} \leftarrow SS.sh(\boldsymbol{r}_i^1, |U_1|, t), \tag{9}$$

$$(u_j, r_{i,j}^2)_{u_j \in U_1} \leftarrow SS.sh(r_i^2, |U_1|, t). \tag{10}$$

Then user u_i uses symmetric encryption to encrypt the secret share:

$$\boldsymbol{e}_{i,j} = Enc\left(KA.agree\left(sk_i, pk_j\right), \boldsymbol{r}_{i,j}^1 || r_{i,j}^2\right), \tag{11}$$

where $Enc(\cdot)$ refers to the symmetric encryption algorithm, and $KA.agree(\cdot)$ refers to the key agreement algorithm.

In addition, the user u_i generates the verification labels of weighted gradient $w_i \boldsymbol{x}_i^T$ and weight w_i as follows:

$$\boldsymbol{V}_i^1 = \boldsymbol{a} \circ (w_i \boldsymbol{x}_i^T + \boldsymbol{r}_i^1) \tag{12}$$

$$V_i^2 = c(w_i + r_i^2) \tag{13}$$

where " $\circ$ " is the Hadamard product [22].

Finally, user u_i sends $(\{u_j, \boldsymbol{e}_{i,j}\}_{u_j \in U_1}, [\![w_i \boldsymbol{x}_i^T]\!], [\![w_i]\!], \boldsymbol{V}_i^1, V_i^2)$ to the cloud server. The set of all users who send above message to the cloud server is denoted as U_2. Note that $U_2 \subset U_1$. The cloud server distributes $\{\boldsymbol{e}_{i,j}\}_{u_i \in U_2}$ to the corresponding users u_j in U_2.

Phase 3: Secure aggregation phase

User u_j receives $\{\boldsymbol{e}_{i,j}\}_{u_i \in U_2}$ from the cloud server, and decrypts them to obtain the corresponding mask shares:

$$\boldsymbol{r}_{i,j}^1 || r_{i,j}^2 = Dec(KA.agree(sk_j, pk_i), \boldsymbol{e}_{i,j}), \tag{14}$$

where $Dec(\cdot)$ refers to the symmetric decryption algorithm.

Then, user u_j calculates $\boldsymbol{R}_j^1 = \sum_{u_i \in U_2} \boldsymbol{r}_{i,j}^1$ and $R_j^2 = \sum_{u_i \in U_2} r_{i,j}^2$, and sends $(\boldsymbol{R}_j^1, R_j^2)$ to the cloud server. The set of all users who send above message to the cloud server is denoted as U_3. Notes that $U_3 \subset U_2$. The cloud server utilizes the homomorphism of the secret sharing to reconstruct the sum of masks $\boldsymbol{r}_i^1$ and r_i^2 as follows:

$$\boldsymbol{R}^1 = \sum\nolimits_{u_i \in U_2} \boldsymbol{r}_i^1 = SS.re(\{(u_j, \boldsymbol{R}_j^1)\}_{u_j \in U_3}, t), \tag{15}$$

$$R^2 = \sum\nolimits_{u_i \in U_2} r_i^2 = SS.re(\{(u_j, R_j^2)\}_{u_j \in U_3}, t). \tag{16}$$

Meanwhile, the cloud server aggregates the weighted gradient and weight as follows:

$$\boldsymbol{X}_{agg} = \sum\nolimits_{u_i \in U_2} [\![w_i \boldsymbol{x}_i^T]\!] - \sum\nolimits_{u_i \in U_2} \boldsymbol{r}_i^1 = \sum\nolimits_{u_i \in U_2} w_i \boldsymbol{x}_i^T + |U_2| \boldsymbol{b}_1 \tag{17}$$

$$W_{agg} = \sum\nolimits_{u_i \in U_2} [\![w_i]\!] - \sum\nolimits_{u_i \in U_2} r_i^2 = \sum\nolimits_{u_i \in U_2} w_i + |U_2| b_2 \tag{18}$$

In addition, the cloud server aggregates the verification labels of all users in U_2 as follows:

$$\boldsymbol{V}_{agg}^1 = \sum\nolimits_{u_i \in U_2} \boldsymbol{V}_i^1 = \sum\nolimits_{u_i \in U_2} \boldsymbol{a} \circ (w_i \boldsymbol{x}_i^T + \boldsymbol{r}_i^1) \tag{19}$$

$$V_{agg}^2 = \sum\nolimits_{u_i \in U_2} V_i^2 = \sum\nolimits_{u_i \in U_2} c(w_i + r_i^2) \tag{20}$$

Finally, the cloud server sends the aggregation results ($\boldsymbol{X}_{agg}$, W_{agg}), verification labels ($\boldsymbol{V}_{agg}^1, V_{agg}^2$), and aggregation masks ($\boldsymbol{R}^1$, R^2) to all users.

Phase 4: Gradient update phase

Users first check that the tuple (sid, T) matches the expected round and has not been seen before, duplicates are rejected. After the users receive above message, they firstly verify the correctness of the aggregation results. That is, users check whether the following two equations hold:

$$\boldsymbol{V}_{agg}^1 = \boldsymbol{a} \circ (\boldsymbol{X}_{agg} - |U_2|\boldsymbol{b}_1 + \boldsymbol{R}^1) \tag{21}$$

$$V_{agg}^2 = c(W_{agg} - |U_2|b_2 + R^2) \tag{22}$$

If above two equations hold, users update the global gradient as $\boldsymbol{x}^{T+1}$ as follows:

$$\boldsymbol{x}^{T+1} = \frac{\boldsymbol{X}_{agg} - |U_2|\boldsymbol{b}_1}{W_{agg} - |U_2|b_2} = \frac{\sum_{u_i \in U_2} w_i \boldsymbol{x}_i^T}{\sum_{u_i \in U_2} w_i}. \tag{23}$$

If the verification fails, abort.

Notably, if the online users are fewer than the threshold t in any time, the scheme aborts. That is, we need $|U| \geq |U_1| \geq |U_2| \geq |U_3| \geq t$. From another perspective, the proposed scheme supports user dropouts. As long as the number of online users in each phase exceeds the threshold t, the cloud server can correctly aggregate the weighted gradients and weights.

6. Security Analysis

This section evaluates the security guarantees of the proposed protocol.

6.1. Data Privacy

Theorem 1. *Under our protocol, neither the cloud server nor any coalition of fewer than t peers can obtain user u_i's weighted gradient $w_i \boldsymbol{x}_i^T$ or its corresponding weight w_i.*

Proof: We work over a finite field $\mathbb{F}$. Secret sharing uses a (t, n) Shamir threshold. For each round T, fresh seeds $b^{(T)} = PRF(B, sid||T)$ are derived from a master seed B. The adversary can control the cloud server and at most $t - 1$ users. Channels from TA to users are secure.

With truly uniform masks $\boldsymbol{r}_i^1 \in \mathbb{F}^m$ and $r_i^2 \in \mathbb{F}$ that are independent of the plaintexts, the ciphertexts $[\![w_i \boldsymbol{x}_i^T]\!]$ and $[\![w_i]\!]$ are distributed uniformly over $\mathbb{F}^m$ and $\mathbb{F}$ and reveal no information about $w_i \boldsymbol{x}_i^T$ or w_i. By the (t, n) property of Shamir secret sharing, any set of fewer than t shares is independent of the secret, so any coalition of size $< t$ learns nothing about individual contributions.

We next replace the outputs of $PRF(B, sid||T)$ with truly uniform seeds in a standard hybrid. By PRF security, the real execution and this hybrid are computationally indistinguishable to any adversary. In the hybrid world the masks are truly random, so the statistical argument above applies directly. Consequently, in the real protocol the adversary's view is computationally indistinguishable from the ideal view that reveals only the aggregates.

Therefore neither the server nor any peer learns $w_i \boldsymbol{x}_i^T$ or w_i beyond the released aggregates.

6.2. Verifiability

Theorem 2. *In our proposed scheme, the cloud server is unable to fabricate aggregation results that would successfully pass user verification.*

Proof: Users compute per-round verification labels using $b^{(T)} = PRF(B, sid||T)$ and the user-held secrets $(\boldsymbol{a}, b, c)$, and the tags are bound to (sid, T). The adversary and may act maliciously as the server.

Assume there exists a server that outputs an aggregate different from the true plaintext sum and still passes all honest users' checks with non-negligible probability. From such a server we obtain either a method to predict a fresh valid verification tag without the user-held secrets, or a method to distinguish PRF outputs from uniform. The former is equivalent to successfully forging the label, while the latter is equivalent to distinguishing between pseudo-random and true random. Both of these are contradictory to the aforementioned assumption. Therefore, any incorrect aggregation will pass the verification only with an negligible probability.

Hence the scheme is sound: forged aggregation results are rejected by honest users except with negligible probability.

6.3. Attack Analysis and Countermeasures

Collusion between the server and clients. Privacy holds against any coalition of fewer than t users even when the server is malicious, because fewer than t Shamir shares are independent of the secret. If $\geq t$ users collude with

the server, privacy can be broken. Hence t should be set no smaller than the minimal cohort size we are willing to reveal at round time, and the protocol aborts whenever $|U_3| < t$. In practice we also require $|U_2| \geq k_{\min}$ $(\geq t)$ before releasing any aggregate.

Replay or tag-reuse by the server. Every ciphertext, tag, and mask is bound to (sid, T), and per-round seeds are fresh. Clients check that the received pair (sid, T) matches the expected round and reject duplicate or out-of-order tuples. Replaying an older $(\boldsymbol{X}_{agg}, W_{agg}, \boldsymbol{V}^1_{agg}, V^2_{agg})$ from round T' cannot pass the checks for $T \neq T'$, because the masks and tags are derived from the new seed $b^{(T)}$ and thus differ.

Leakage when few participants remain. Aggregates computed from very small cohorts can leak via differencing. We therefore enforce a minimum cohort size and weight dispersion policy: if $|U_2| < k_{\min}$ or if $\max_i w_i$ is too large relative to the total, the round aborts. Concretely, for a chosen $0 < \rho < 1$, require

$$\max_{u_i \in U_2} w_i \;\leq\; \rho \cdot \sum_{u_i \in U_2} w_i \quad \text{and} \quad |U_2| \geq k_{\min}\ (\geq t).$$

This prevents any single user (or tiny set) from dominating the weighted sum.

Reuse of random seeds across rounds. To avoid cross-round correlation, per-round values are derived with domain separation:

$$b^{(T)} = PRF(B,\, sid \,||\, T), \boldsymbol{b}_1 = PRF(b^{(T)},\, \texttt{"mask:x"}), b_2 = PRF(b^{(T)},\, \texttt{"mask:w"}).$$

No seed is reused across rounds or purposes. If a client restarts, it recomputes the same $b^{(T)}$ from (B, sid, T) and continues; if the session restarts, a fresh sid is chosen.

7. Experiments

All simulations were executed on a Windows 11 workstation equipped with an Intel Core i5-9300H @ 2.40 GHz, 16 GB RAM, and an NVIDIA GeForce GTX 1650 GPU. The cryptographic primitives are elliptic-curve Diffie-Hellman and a standard (t, n)-threshold Shamir scheme, both implemented in Python. We adopt the MNIST handwritten-digit dataset (60 000 training & 10 000 test samples) and distribute it to $N = 100$ clients in an independent and identically distributed (IID) manner. Each client trains a

4-layer convolutional neural network consisting of two 32-channel 3×3 conv-ReLU layers followed by 2×2 max-pooling, then two 64-channel conv-ReLU layers with another max-pooling, and finally two fully-connected layers of sizes 128 and 10. Unless stated otherwise, every local update is run for one epoch with a learning rate of 0.01 and a batch size of 10, and fix the secret-sharing threshold at $t = 0.8N$, allowing 20% of the clients to drop out without aborting the round. Low-quality data are simulated by randomly flipping labels.

7.1. Functionality

This section takes the functional angle to position our scheme against five representative baselines listed in Table 1. Starting from the classical secure-aggregation line, PPML [23] protects gradient privacy with double masking and Shamir secret sharing while tolerating client drop-out, yet it inherits the implicit assumption that every participant owns high-quality, IID data and therefore fails to handle unreliable users. SecProbe [6] moves one step forward by explicitly filtering those unreliable clients through a differential-privacy top-k mechanism; the price, however, is that the round aborts once any selected top-k device disconnects, leaving the protocol fragile to drop-out.

To mitigate noisy data without losing drop-out tolerance, PPFDL [7] replaces differential privacy filtering with homomorphic encryption plus truth-discovery weighting. This design successfully down-weights bad gradients, but it relies on two interoperating servers and still trusts the cloud to return a correct result. ESFL [24] addresses that very trust issue: it augments single-mask aggregation with one-time random tags so every client can locally verify the server's computation at negligible cost—yet because it keeps equal weights, the method remains sensitive to low-quality inputs. Pushing verifiability further, VeriFL [10] attaches a zero-knowledge proof to each aggregation, achieving strong soundness but inflating communication by an order of magnitude.

In contrast to the above approaches, we eliminate the negative impact of low-quality data on the global model. Specifically, leveraging the additive homomorphic property of secret sharing, a secure aggregation protocol is constructed to address the problem of untrustworthy cloud servers while ensuring privacy and security.

Table 1: Functional comparison with previous schemes (DP = Differential Privacy; Tag = one-time validation tag; ZKP = zero-knowledge proof; LQ-Robust = robustness to low-quality clients; Drop-out Tol. = tolerance to client drop-out)

Scheme	Privacy-Preserving	Verifiable	LQ-Robust	Drop-out Tol.	Single-server	Technique
PPML [23]	✓	×	×	✓	✓	Double-mask/ Shamir
SecProbe [6]	✓	×	✓	×	✓	DP Noise/ Top-k Filter
PPFDL [7]	✓	×	✓	×	×	Paillier/ Truth Discovery
ESFL [24]	✓	✓	×	×	✓	Single-mask/ Tag
VeriFL [10]	✓	✓	×	×	✓	Double-mask/ ZKP
Ours	✓	✓	✓	✓	✓	Double-mask/ Secret Sharing / Truth Discovery

7.2. Accuracy

To evaluate robustness against low-quality clients, we deliberately compare our method with only ESFL [24] and PPFDL [7]—the two baselines that share the same secret-sharing, single-round aggregation backbone but differ precisely in their handling of unreliable data. ESFL keeps every client equally weighted and thus represents the "no-robustness" reference, whereas PPFDL applies the same χ^2-based weighting rule as ours but relies on a Paillier homomorphic-encryption protocol executed jointly by two non-colluding servers. We omit PPML and SecProbe here because neither scheme applies any weighting; their accuracy would overlap with ESFL when $P = 0$ and drop even faster once noisy clients are introduced.

All subsequent experiments use MNIST, a 4-Conv-2-FC CNN, 100 clients, 100 rounds, and a learning rate of 0.01. We randomly flip the labels of P% of clients to mimic low-quality data, where $P \in 0, 0.05 \ldots, 0.30$, each configuration is repeated five times, and Fig. 2 plots the mean accuracy with one-standard-deviation error bars. When $P = 0$ all three methods converge to approximately 0.98. As P rises, ESFL plummets to 0.517 at $P = 0.30$ because unweighted noisy clients dominate the update. By contrast, PPFDL and our scheme remain above 0.89; our single-server variant even achieves 0.90 without discarding any round.

These results show that χ^2 weighting is essential for noisy environments; it lifts accuracy by more than 37 percentage points over ESFL at $P = 0.30$. Moreover, our design matches PPFDL-level robustness while avoiding its two-server requirement and large Paillier ciphertexts, thus providing the best accuracy-to-deployment trade-off among the compared approaches.

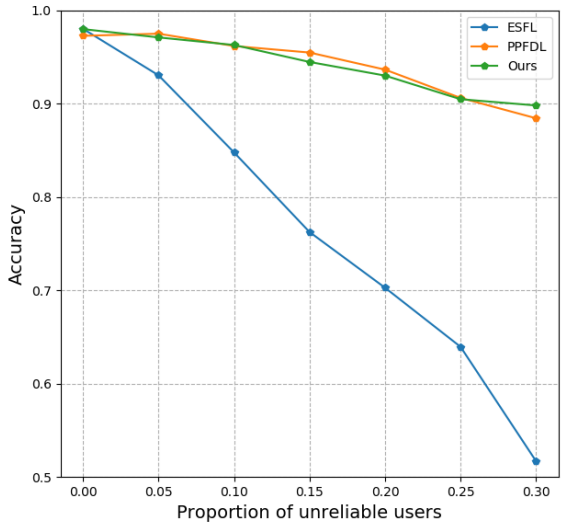

Figure 2: Test accuracy versus the proportion P of low-quality clients. (Each point is averaged over five independent runs; error bars indicate one standard deviation).

7.3. Efficiency

To measure the cost introduced solely by verifiability, we reuse the MNIST setting of Sec.7.2, 4-Conv-2-FC CNN, 100–500 clients, 100 rounds, learning rate 0.01, batch size 10—but switch off the χ^2-weighting module. Two single-server baselines are retained for comparison: ESFL [24] (one-time tag) and VeriFL [10] (Bulletproof proof). PPML and SecProbe are omitted here because they contain no verification layer, so their traffic equals the plaintext baseline. For each N we record (i) per-client uplink bytes and (ii) server CPU time per round.

Figure 3 shows that our scheme and ESFL overlap at approximately 0.95 MB when $N = 100$ and 0.98 MB when $N = 500$; the extra cost is merely a 16-byte tag, and server latency stays below 5 ms. By contrast, VeriFL carries an additional approximately 640 kB, raising traffic to 1.6-1.7 MB and increasing proof-generation time from 0.12 s (100 clients) to 0.40 s (500 clients).

Hence, with the same bandwidth and latency as ESFL, our design removes ESFL's robustness gap (Sec.7.2) while avoiding VeriFL's heavy zero-knowledge burden. When the χ^2 weights are re-enabled, the identical communication budget yields markedly higher accuracy under noisy clients, giving our scheme the best trade-off among all evaluated single-server methods.

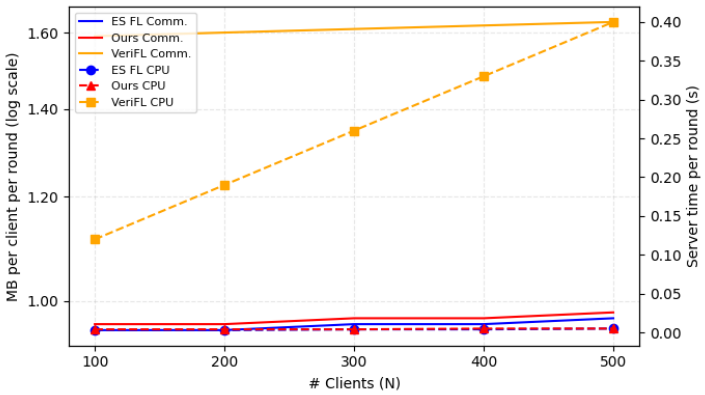

Figure 3: Per-Round Communication Cost and Server CPU Overhead

8. Conclusion

We present a verifiable secure aggregation approach in privacy-preserving federated learning to address the issues of low-quality data. Specifically, our approach utilizes truth discovery to prioritize the contributions of different quality data, thus reducing low-quality data's influence on the final aggregated model. Additionally, we introduce an aggregation technique that employs a two-layer mask to safeguard user privacy and prevent inadvertent information disclosure. Users can independently check that the aggregation is correct to mitigate the risk of erroneous results from the cloud server. Security analysis confirms the assurance of privacy when the number of colluding users is fewer than $t-1$. Experimental results show that our scheme outperforms existing methods in both training accuracy and efficiency. However, as our system requires multiple rounds of interactions to complete an entire iteration, it experiences communication delay. Moreover, we mainly focused on eliminating low-quality data under the scenario of semi-honest users and did not further take malicious users into account. In future work, we will explore how to curb the adverse effects of low-quality inputs within this system architecture, realize verifiable, privacy-preserving federated learning with a single server, and reduce communication time and computational cost.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work

reported in this paper.

Ethical Statement

No conflict of interest exists in the submission of this manuscript, and the manuscript is approved by all authors for publication. We would like to declare that the work described was original research that has not been published previously, and is not under consideration for publication elsewhere, in whole or in part.

Acknowledgments

We acknowledge the partial support of this research by the National Natural Science Foundation of China under Grant 12401663.

References

[1] Michelle Goddard. The EU General Data Protection Regulation (GDPR): European regulation that has a global impact. *International Journal of Market Research*, 59(6):703–705, 2017.

[2] H Brendan McMahan, FX Yu, P Richtarik, AT Suresh, D Bacon, et al. Federated learning: Strategies for improving communication efficiency. In *Proceedings of the 29th Conference on Neural Information Processing Systems (NIPS), Barcelona, Spain*, pages 5–10, 2016.

[3] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 3–18, 2017.

[4] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. Model inversion attacks that exploit confidence information and basic countermeasures. In *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, pages 1322–1333, 2015.

[5] Yiran Li, Hongwei Li, Guowen Xu, Xiaoming Huang, and Rongxing Lu. Efficient privacy-preserving federated learning with unreliable users. *IEEE Internet of Things Journal*, 9(13):11590–11603, 2022.

[6] Lingchen Zhao, Qian Wang, Qin Zou, Yan Zhang, and Yanjiao Chen. Privacy-preserving collaborative deep learning with unreliable participants. *IEEE Transactions on Information Forensics and Security*, 15:1486–1500, 2020.

[7] Guowen Xu, Hongwei Li, Yun Zhang, Shengmin Xu, Jianting Ning, and Robert H. Deng. Privacy-preserving federated deep learning with irregular users. *IEEE Transactions on Dependable and Secure Computing*, 19(2):1364–1381, 2022.

[8] Changhee Hahn, Hodong Kim, Minjae Kim, and Junbeom Hur. Versa: Verifiable secure aggregation for cross-device federated learning. *IEEE Transactions on Dependable and Secure Computing*, 20(1):36–52, 2023.

[9] Guowen Xu, Hongwei Li, Sen Liu, Kan Yang, and Xiaodong Lin. Verifynet: Secure and verifiable federated learning. *IEEE Transactions on Information Forensics and Security*, 15:911–926, 2020.

[10] Xiaojie Guo, Zheli Liu, Jin Li, Jiqiang Gao, Boyu Hou, Changyu Dong, and Thar Baker. Verifl: Communication-efficient and fast verifiable aggregation for federated learning. *IEEE Transactions on Information Forensics and Security*, 16:1736–1751, 2021.

[11] Anmin Fu, Xianglong Zhang, Naixue Xiong, Yansong Gao, Huaqun Wang, and Jing Zhang. Vfl: A verifiable federated learning with privacy-preserving for big data in industrial iot. *IEEE Transactions on Industrial Informatics*, 18(5):3316–3326, 2022.

[12] Yi Xu, Changgen Peng, Weijie Tan, Youliang Tian, Minyao Ma, and Kun Niu. Non-interactive verifiable privacy-preserving federated learning. *Future Generation Computer Systems*, 128:365–380, 2022.

[13] Jiewang Cai, Wenting Shen, and Jing Qin. Esvfl: Efficient and secure verifiable federated learning with privacy-preserving. *Information Fusion*, 109:102420, 2024.

[14] Yuanjun Xia, Yining Liu, Shi Dong, Meng Li, and Cheng Guo. Svca: Secure and verifiable chained aggregation for privacy-preserving federated learning. *IEEE Internet of Things Journal*, 11(10):18351–18365, 2024.

[15] Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloe Kiddon, Jakub Konečnỳ, Stefano Mazzocchi, H. Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and Jason Roselander. Towards federated learning at scale: System design. In *Proceedings of the 2nd SysML (MLSys) Conference*, 2019. Production system design for cross-device FL.

[16] Micah J. Sheller, Brandon Edwards, G. Anthony Reina, Jason Martin, Sarthak Pati, Aikaterini Kotrotsou, Mikhail Milchenko, Weilin Xu, Daniel Marcus, Rivka R. Colen, and Spyridon Bakas. Federated learning in medicine: facilitating multi-institutional collaborations without sharing patient data. *Scientific Reports*, 10(12598), 2020.

[17] Ittai Dayan, Holger R. Roth, Aoxiao Zhong, et al. Federated learning for predicting clinical outcomes in patients with covid-19. *Nature Medicine*, 27:1735–1743, 2021.

[18] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. PMLR, 2017.

[19] Chenglin Miao, Wenjun Jiang, Lu Su, Yaliang Li, Suxin Guo, Zhan Qin, Houping Xiao, Jing Gao, and Kui Ren. Cloud-enabled privacy-preserving truth discovery in crowd sensing systems. In *Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems*, pages 183–196, 2015.

[20] Oded Goldreich. Secure multi-party computation. *Manuscript. Preliminary version*, 78(110):1–108, 1998.

[21] Jieyu Xu, Hongwei Li, Jia Zeng, and Meng Hao. Efficient and privacy-preserving federated learning with irregular users. In *ICC 2022 - IEEE International Conference on Communications*, pages 534–539, 2022.

[22] Elizabeth Million. The hadamard product. *Course Notes*, 3(6):1–7, 2007.

[23] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and

Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1175–1191, 2017.

[24] Zhen Yang, Ming Zhou, Haiyang Yu, Richard O. Sinnott, and Huan Liu. Efficient and secure federated learning with verifiable weighted average aggregation. *IEEE Transactions on Network Science and Engineering*, 10(1):205–222, 2023.